

Упражнение 1: Поиск минимального и максимального элемента

```
#include <iostream>
using namespace std;

int main() {
    // Создаем массив из 5 целых чисел
    int arr[5] = {42, 15, 8, 63, 19};

    // Инициализируем переменные для хранения минимума и максимума
    // Начинаем с первого элемента массива
    int min = arr[0];
    int max = arr[0];

    // Перебираем все элементы массива, начиная со второго (и
    for(int i = 1; i < 5; i++) {
        // Если текущий элемент меньше текущего минимума
        if(arr[i] < min) {
            min = arr[i]; // обновляем значение минимума
        }

        // Если текущий элемент больше текущего максимума
        if(arr[i] > max) {
            max = arr[i]; // обновляем значение максимума
        }
    }

    // Выводим результаты
    cout << "Массив: ";
    for(int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    cout << "Минимальный элемент: " << min << endl;
    cout << "Максимальный элемент: " << max << endl;
```

```
    return 0;  
}
```

Упражнение 2: Сортировка пузырьком

```
#include <iostream>  
using namespace std;  
  
int main() {  
    // Исходный неотсортированный массив  
    int arr[6] = {64, 34, 25, 12, 22, 11};  
  
    cout << "Исходный массив: ";  
    for(int i = 0; i < 6; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
  
    // Внешний цикл: проходим по массиву (n-1) раз  
    // Каждый проход "всплывает" самый большой элемент в конец  
    for(int i = 0; i < 5; i++) { // 5 = 6-1 (длина массива - 1)  
  
        // Внутренний цикл: сравниваем соседние элементы  
        // После каждого прохода внешнего цикла самый большой  
        // уже находится на своем месте, поэтому проверяем на  
        for(int j = 0; j < 5 - i; j++) {  
  
            // Если текущий элемент больше следующего  
            if(arr[j] > arr[j + 1]) {  
                // Меняем их местами  
                int temp = arr[j];        // Сохраняем текущий элемент  
                arr[j] = arr[j + 1];      // На место текущего ставим следующий  
                arr[j + 1] = temp;        // На место следующего ставим текущий  
            }  
        }  
  
        // Можно вывести промежуточное состояние массива после  
        cout << "После прохода " << i + 1 << ": ";  
        for(int k = 0; k < 6; k++) {  
            cout << arr[k] << " ";  
        }  
    }  
}
```

```
        cout << endl;
    }

    cout << "Отсортированный массив: ";
    for(int i = 0; i < 6; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Упражнение 3: Подсчет суммы и среднего значения

```
#include <iostream>
using namespace std;

int main() {
    // Массив с оценками студента
    int grades[7] = {5, 4, 3, 5, 4, 5, 4};

    // Переменная для хранения суммы
    int sum = 0;

    // Переменная для хранения минимальной оценки
    int minGrade = grades[0];

    // Переменная для хранения максимальной оценки
    int maxGrade = grades[0];

    // Проходим по всем элементам массива
    for(int i = 0; i < 7; i++) {
        // Добавляем текущую оценку к сумме
        sum = sum + grades[i];

        // Проверяем, является ли текущая оценка новой минимал
        if(grades[i] < minGrade) {
            minGrade = grades[i];
        }
    }
}
```

```
    }

    // Проверяем, является ли текущая оценка новой максимальной
    if(grades[i] > maxGrade) {
        maxGrade = grades[i];
    }
}

// Вычисляем среднее значение
// Важно: преобразуем sum к типу double для точного деления
double average = (double)sum / 7;

// Выводим результаты
cout << "Все оценки: ";
for(int i = 0; i < 7; i++) {
    cout << grades[i] << " ";
}
cout << endl;

cout << "Сумма всех оценок: " << sum << endl;
cout << "Средний балл: " << average << endl;
cout << "Минимальная оценка: " << minGrade << endl;
cout << "Максимальная оценка: " << maxGrade << endl;

return 0;
}
```

Упражнение 4: Поиск элемента в массиве

```
#include <iostream>
using namespace std;

int main() {
    // Массив с числами
    int numbers[8] = {23, 45, 67, 12, 89, 34, 56, 78};

    // Число, которое будем искать
    int searchNumber;

    // Переменная для хранения индекса найденного элемента
    // -1 означает, что элемент еще не найден
```

```
int foundIndex = -1;

// Переменная для хранения минимального значения в массиве
int minValue = numbers[0];
// Индекс минимального значения
int minIndex = 0;

cout << "Массив: ";
for(int i = 0; i < 8; i++) {
    cout << numbers[i] << " ";

    // Заодно ищем минимальный элемент и его индекс
    if(numbers[i] < minValue) {
        minValue = numbers[i];
        minIndex = i;
    }
}
cout << endl;

// Выводим информацию о минимальном элементе
cout << "Минимальный элемент: " << minValue << endl;
cout << "Его индекс в массиве: " << minIndex << endl;

// Запрашиваем у пользователя число для поиска
cout << "Введите число для поиска: ";
cin >> searchNumber;

// Ищем число в массиве
for(int i = 0; i < 8; i++) {
    if(numbers[i] == searchNumber) {
        foundIndex = i; // Запоминаем индекс найденного эл
        break; // Прерываем цикл, так как элемент найден
    }
}

// Выводим результат поиска
if(foundIndex != -1) {
    cout << "Число " << searchNumber << " найдено на позиц
} else {
    cout << "Число " << searchNumber << " не найдено в мас
}
```

```
    return 0;  
}
```

Задача 5: Обратный порядок элементов массива

```
#include <iostream>  
using namespace std;  
  
int main() {  
    // Создаем массив и определяем его размер  
    int arr[] = {1, 2, 3, 4, 5, 6, 7};  
    int size = 7; // размер массива  
  
    cout << "Исходный массив: ";  
    for(int i = 0; i < size; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
  
    // Алгоритм обращения массива:  
    // Меняем местами первый с последним, второй с предпоследним  
    // Идем только до середины массива, иначе вернемся к исходному  
    for(int i = 0; i < size / 2; i++) {  
        // Сохраняем значение левого элемента  
        int temp = arr[i];  
        // Заменяем левый элемент правым  
        arr[i] = arr[size - 1 - i];  
        // Заменяем правый элемент сохраненным левым  
        arr[size - 1 - i] = temp;  
  
        // size - 1 - i - это индекс элемента с конца:  
        // size - 1 - последний элемент  
        // size - 1 - i - элемент, отстоящий на i позиций от конца  
    }  
  
    cout << "Перевернутый массив: ";  
    for(int i = 0; i < size; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;
```

```
    return 0;  
}
```

Задача 6: Подсчет положительных, отрицательных и нулевых элементов

```
#include <iostream>  
using namespace std;  
  
int main() {  
    // Массив с разными числами  
    int numbers[10] = {5, -3, 0, 12, -7, 0, 8, -1, 4, -9};  
  
    // Счетчики для разных типов чисел  
    int positiveCount = 0; // счетчик положительных чисел  
    int negativeCount = 0; // счетчик отрицательных чисел  
    int zeroCount = 0;     // счетчик нулей  
  
    cout << "Анализируемый массив: ";  
    for(int i = 0; i < 10; i++) {  
        cout << numbers[i] << " ";  
  
        // Проверяем тип текущего числа  
        if(numbers[i] > 0) {  
            positiveCount++; // увеличиваем счетчик положительных  
        }  
        else if(numbers[i] < 0) {  
            negativeCount++; // увеличиваем счетчик отрицательных  
        }  
        else {  
            zeroCount++; // увеличиваем счетчик нулей  
        }  
    }  
    cout << endl;  
  
    // Выводим статистику  
    cout << "\nСтатистика по массиву:" << endl;  
    cout << "Положительных чисел: " << positiveCount << endl;  
    cout << "Отрицательных чисел: " << negativeCount << endl;
```

```
cout << "Нулей: " << zeroCount << endl;

// Дополнительно: находим сумму всех положительных чисел
int positiveSum = 0;
for(int i = 0; i < 10; i++) {
    if(numbers[i] > 0) {
        positiveSum += numbers[i];
    }
}
cout << "Сумма положительных чисел: " << positiveSum << endl;

return 0;
}
```

Задача 7: Удаление дубликатов из массива

```
#include <iostream>
using namespace std;

int main() {
    // Массив с повторяющимися элементами
    int arr[12] = {1, 2, 3, 2, 4, 5, 1, 6, 3, 7, 8, 5};
    int size = 12;

    cout << "Исходный массив (" << size << " элементов): ";
    for(int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // Создаем новый массив для уникальных элементов
    // Максимальный размер - как у исходного массива
    int uniqueArr[12];
    int uniqueCount = 0; // счетчик уникальных элементов

    // Проходим по всем элементам исходного массива
    for(int i = 0; i < size; i++) {
        bool isDuplicate = false; // флаг, указывающий на дубликат

        // Проверяем, есть ли текущий элемент уже в массиве уникальных
        for(int j = 0; j < uniqueCount; j++) {
```



```

        if(arr[i] == uniqueArr[j]) {
            isDuplicate = true; // нашли дубликат
            break; // прерываем внутренний цикл
        }
    }

    // Если элемент не дубликат, добавляем его в массив у
    if(!isDuplicate) {
        uniqueArr[uniqueCount] = arr[i];
        uniqueCount++; // увеличиваем счетчик уникальных
    }
}

cout << "\nМассив без дубликатов (" << uniqueCount << " эл
for(int i = 0; i < uniqueCount; i++) {
    cout << uniqueArr[i] << " ";
}
cout << endl;

return 0;
}

```

Задача 8: Объединение двух массивов

```

#include <iostream>
using namespace std;

int main() {
    // Два исходных массива
    int arr1[4] = {10, 20, 30, 40};
    int arr2[5] = {50, 60, 70, 80, 90};

    // Вычисляем размеры массивов
    int size1 = 4;
    int size2 = 5;

    // Создаем массив для результата (сумма размеров двух мас
    int result[9]; // 4 + 5 = 9
    int resultSize = size1 + size2;

    cout << "Первый массив: ";
}

```

```
for(int i = 0; i < size1; i++) {
    cout << arr1[i] << " ";
}
cout << endl;

cout << "Второй массив: ";
for(int i = 0; i < size2; i++) {
    cout << arr2[i] << " ";
}
cout << endl;

// Копируем элементы из первого массива
for(int i = 0; i < size1; i++) {
    result[i] = arr1[i];
}

// Копируем элементы из второго массива
// Начинаем с позиции size1 (после последнего элемента пер
for(int i = 0; i < size2; i++) {
    result[size1 + i] = arr2[i];
}

cout << "\nОбъединенный массив: ";
for(int i = 0; i < resultSize; i++) {
    cout << result[i] << " ";
}
cout << endl;

// Дополнительно: сортируем объединенный массив пузырьком
for(int i = 0; i < resultSize - 1; i++) {
    for(int j = 0; j < resultSize - 1 - i; j++) {
        if(result[j] > result[j + 1]) {
            // Меняем местами
            int temp = result[j];
            result[j] = result[j + 1];
            result[j + 1] = temp;
        }
    }
}

cout << "Отсортированный объединенный массив: ";
for(int i = 0; i < resultSize; i++) {
    cout << result[i] << " ";
}
```

```
    }  
    cout << endl;  
  
    return 0;  
}
```

Задача 9: Поиск наиболее часто встречающегося элемента (моды)

```
#include <iostream>  
using namespace std;  
  
int main() {  
    // Массив для анализа  
    int arr[15] = {2, 3, 4, 2, 5, 3, 2, 6, 2, 7, 8, 2, 9, 3, 4};  
    int size = 15;  
  
    // Переменные для хранения информации о самом частом элементе  
    int mostFrequent = arr[0]; // значение самого частого элемента  
    int maxFrequency = 0;      // максимальная частота  
  
    cout << "Анализируемый массив: ";  
    for(int i = 0; i < size; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl << endl;  
  
    // Проверяем каждый элемент массива  
    for(int i = 0; i < size; i++) {  
        int currentElement = arr[i];  
        int currentFrequency = 0;  
  
        // Считаем, сколько раз currentElement встречается в массиве  
        for(int j = 0; j < size; j++) {  
            if(arr[j] == currentElement) {  
                currentFrequency++;  
            }  
        }  
  
        // Выводим информацию о текущем элементе
```

```
cout << "Элемент " << currentElement << " встречается  
    << currentFrequency << " раз" << endl;  
  
    // Если текущая частота больше максимальной,  
    // обновляем информацию о самом частом элементе  
    if(currentFrequency > maxFrequency) {  
        maxFrequency = currentFrequency;  
        mostFrequent = currentElement;  
    }  
}  
  
cout << "\nРезультат анализа:" << endl;  
cout << "Самый частый элемент: " << mostFrequent << endl;  
cout << "Он встречается " << maxFrequency << " раз" << endl;  
  
// Дополнительно: проверяем, есть ли несколько элементов с  
cout << "\nЭлементы, встречающиеся " << maxFrequency << " раз:  
bool checked[15] = {false}; // массив для отслеживания появления  
  
for(int i = 0; i < size; i++) {  
    // Пропускаем, если элемент уже проверяли  
    if(checked[i]) continue;  
  
    int count = 0;  
    // Считаем частоту текущего элемента  
    for(int j = 0; j < size; j++) {  
        if(arr[j] == arr[i]) {  
            count++;  
            checked[j] = true; // помечаем как проверенный  
        }  
    }  
  
    // Если частота равна максимальной, выводим элемент  
    if(count == maxFrequency) {  
        cout << arr[i] << " ";  
    }  
}  
cout << endl;  
  
return 0;  
}
```

Задача 10: Циклический сдвиг массива

```
#include <iostream>
using namespace std;

int main() {
    int arr[8] = {1, 2, 3, 4, 5, 6, 7, 8};
    int size = 8;

    cout << "Исходный массив: ";
    for(int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // Запрашиваем у пользователя величину сдвига
    int shift;
    cout << "Введите величину сдвига (положительное - вправо,
    cin >> shift;

    // Нормализуем сдвиг (если shift больше размера массива)
    shift = shift % size;

    // Если сдвиг отрицательный, преобразуем его в эквивалентный
    if(shift < 0) {
        shift = size + shift;
    }

    cout << "Выполняем циклический сдвиг на " << shift << " п

    // Создаем временный массив для хранения сдвинутого резул
    int tempArr[8];

    // Выполняем циклический сдвиг
    for(int i = 0; i < size; i++) {
        // Вычисляем новую позицию для элемента
        // % size обеспечивает циклический переход (с конца в
        int newPosition = (i + shift) % size;
        tempArr[newPosition] = arr[i];
    }

    // Копируем результат обратно в исходный массив
```

```
for(int i = 0; i < size; i++) {  
    arr[i] = tempArr[i];  
}  
  
cout << "Массив после сдвига: ";  
for(int i = 0; i < size; i++) {  
    cout << arr[i] << " ";  
}  
cout << endl;  
  
return 0;  
}
```

Дополнительные задания для практики:

1. **Напишите программу**, которая находит все простые числа в массиве.
2. **Создайте программу** для умножения всех элементов массива на заданное число.
3. **Реализуйте алгоритм** поиска двух наибольших элементов в массиве.
4. **Напишите программу**, которая разделяет массив на два: с четными и нечетными числами.
5. **Создайте игру "Угадай число"**, где компьютер загадывает число, а массив хранит историю попыток пользователя.

Советы по решению задач:

1. **Сначала продумайте алгоритм** на бумаге или в уме, прежде чем писать код.
2. **Используйте псевдокод** для описания шагов решения.
3. **Тестируйте на маленьких массивах**, где можно легко проверить результат вручную.
4. **Добавляйте промежуточные выводы** для отладки сложных алгоритмов.

5. Пробуйте модифицировать задачи - например, сделать функцию для каждой операции.

Эти задачи помогут закрепить понимание работы с массивами и подготовят к решению более сложных алгоритмических задач!